

Installing and Configuring OpenCode

A Student Guide to CLI-First AI Development

Speedykom Group GmbH

2026-04-16

Introduction

OpenCode is an open-source AI coding agent that runs in your terminal. This guide walks you through installing it on your laptop and teaches you how to extend its capabilities using custom skills. Unlike many AI tools that require expensive subscriptions, OpenCode connects to various model providers—including free options—giving you flexibility and control.

Why Use OpenCode?

- **Privacy-first:** Your code and context data stay on your machine
- **Multi-session support:** Run multiple agents in parallel on the same project
- **Flexible model options:** Use free models, GitHub Copilot, or connect to 75+ providers
- **Terminal-native:** Works entirely in your command line interface
- **Extensible:** Create custom skills to teach the AI domain-specific knowledge

Installing OpenCode

System Requirements

Before installing, ensure you have:

- A terminal/command prompt
- Internet connection
- Administrator/sudo access (for installation only)
- Git installed (for version control features)

Installation Methods

macOS

macOS users have two options: using Homebrew (recommended) or the install script.

Option 1: Homebrew (Recommended)

Open your Terminal (you can find it in Applications > Utilities > Terminal, or press Cmd+Space and type “Terminal”):

```
brew tap anomaly/opencode
brew install opencode
```

Option 2: Install Script

In your Terminal:

```
curl -fsSL https://opencode.ai/install | bash
```

The install script detects your system architecture (Intel or Apple Silicon) and downloads the appropriate binary. After installation, add OpenCode to your PATH:

```
export PATH="$HOME/.local/bin:$PATH"
```

Add this line to your ~/.zshrc or ~/.bash_profile to make it permanent. To do this, run:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
```

Then restart your Terminal or run `source ~/.zshrc`.

Linux

Linux installation uses the same install script approach. The script supports major distributions including Ubuntu, Debian, Fedora, and Arch.

Universal Install Script:

Open your Terminal (press Ctrl+Alt+T on Ubuntu/Debian, or find “Terminal” in your applications menu):

```
curl -fsSL https://opencode.ai/install | bash
```

The installer will:

1. Detect your architecture (x86_64, ARM64)
2. Download the appropriate binary
3. Install to ~/.local/bin/opencode

Verify installation:

In your Terminal, run:

```
opencode --version
```

Add to PATH permanently:

For Bash users (most common on Ubuntu/Debian):

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

For Zsh users (common on newer systems):

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Windows

Windows users can install via WinGet, the official Windows package manager.

Using WinGet (Recommended):

Open PowerShell as Administrator (right-click on Start menu → “Windows PowerShell (Admin)” or “Terminal (Admin)”):

```
winget install Anomaly.OpenCode
```

After installation, restart your PowerShell window. Verify the installation:

```
opencode --version
```

Alternative: Install Script

If WinGet is unavailable, use the curl-based installer in PowerShell (as Administrator):

```
curl -fsSL https://opencode.ai/install | bash
```

Note for Windows users: The WinGet method is preferred because it handles updates automatically through `winget upgrade`. The curl script works but requires manual updates.

First-Time Setup

After installation, configure OpenCode. Open your Terminal (macOS/Linux) or PowerShell (Windows) and run:

opencode configure

This command guides you through:

1. **Selecting a model provider:** Choose from free models (like those available through OpenRouter), or connect your own API keys for OpenAI, Anthropic, or others
2. **Setting default behaviors:** Configure whether to auto-execute commands or ask for confirmation
3. **Git integration:** Optionally link your GitHub account for enhanced features

Connecting to OpenRouter (Recommended for Students)

What is OpenRouter?

OpenRouter is a unified API gateway that gives you access to hundreds of AI models through a single interface. Instead of signing up for multiple services (OpenAI, Anthropic, Google, etc.), you create one OpenRouter account and get access to:

- **300+ models** from 60+ providers (GPT-4, Claude, Gemini, Llama, and more)
- **Free tier** with generous limits for learning and experimentation
- **Pay-as-you-go** pricing with better rates than going direct to providers
- **Higher availability** with automatic failover between providers
- **No subscriptions** required to get started

Why Choose OpenRouter?

Feature	OpenRouter	Direct API (OpenAI/Anthropic)
Cost	Free tier available + pay-per-use	Requires paid API key
Models	Access to 300+ models from all major providers	Limited to that provider only
Setup	One API key for everything	Separate accounts for each provider
Flexibility	Switch models instantly	Locked into one provider
Best for	Students, experimenting, learning	Production with specific models

For students: OpenRouter is ideal because you can start free, experiment with different models, and only pay if you exceed the free tier limits.

Getting Your OpenRouter API Key

Step 1: Create an OpenRouter Account

1. Go to <https://openrouter.ai/>
2. Click “Sign Up” (top right corner)
3. Choose your preferred sign-in method:
 - **Google** (recommended - quickest)
 - **GitHub**
 - **MetaMask** (if you have a crypto wallet)

Step 2: Get Your API Key

1. Go to <https://openrouter.ai/settings/keys> (or click “Settings” → “Keys” in the menu)
2. Click the “Create Key” button
3. Give your key a name (e.g., “OpenCode-Laptop”)
4. Copy the API key (it looks like `sk-or-v1-...`)

Important: Treat this key like a password. Never share it or commit it to public repositories.

Step 3: Connect OpenCode to OpenRouter

Open your Terminal (macOS/Linux) or PowerShell (Windows) and run:

```
opencode configure
```

When prompted to select a model provider: 1. Choose “OpenRouter” from the list 2. Paste your API key when asked 3. Select your preferred model (we recommend starting with a free model like `openai/gpt-4o-mini` or `anthropic/claude-3-haiku`) 4. Save the configuration

Test your connection:

Open your Terminal/PowerShell and type:

```
opencode
```

Once the OpenCode CLI opens, simply type:

```
Hello, are you connected to OpenRouter?
```

If you get a response, you’re all set!

Free vs Paid Models

OpenRouter offers both free and paid models:

Free Models (great for learning): - `openai/gpt-4o-mini` - Fast and capable - `google/gemini-flash-1.5` - Good for longer conversations - `meta-llama/llama-3.1-8b-instruct` - Open source option

Paid Models (when you need more power): - `anthropic/claude-opus-4` - Best for complex coding tasks - `openai/gpt-4o` - Great all-rounder - `google/gemini-2.5-pro` - Excellent for reasoning

Pro Tip: Start with free models. They're surprisingly capable and perfect for learning. Only upgrade to paid models when you hit limitations or need specific capabilities.

Managing Your OpenRouter Account

- **Dashboard:** <https://openrouter.ai/> - View usage, manage keys, browse models
- **Credits:** Add credits under "Billing" if you want to use paid models
- **Model Browser:** <https://openrouter.ai/models> - Explore all available models with pricing

OpenCode Zen: Free Access to Premium Chinese Models

What is OpenCode Zen?

OpenCode Zen is OpenCode's curated model service that provides access to high-quality AI models, including powerful Chinese models like **MinMax** and **Kimi K2.5**, often with **free tiers** or significantly reduced costs compared to direct API access.

Why Use OpenCode Zen?

- **Free tiers available** for models that normally require payment
- **No separate API keys** needed for many models
- **Curated selection** of high-performing models tested by OpenCode
- **Simplified pricing** with straightforward pay-as-you-go options
- **Access to Chinese models** like MinMax M2.5 and Kimi K2.5 that rival GPT-4 and Claude

Available Models in OpenCode Zen

Free/Cheap Tier Models:

MinMax M2.5 (Highly Recommended) - 80.2% on SWE-Bench benchmarks - Excellent for complex coding tasks - Costs around \$1/hour of usage - Great Claude Opus alternative

Kimi K2.5 by Moonshot AI - 1 trillion parameter Mixture-of-Experts model - Activates 32 billion parameters per step - Excellent at reasoning, coding, and visual understanding - Supports up to 100 parallel sub-agents - Open-weight model (you can download and run locally)

Other Chinese Models: - Various models from Alibaba, Baidu, and other Chinese AI labs - Often cheaper than Western equivalents with similar performance

How to Access OpenCode Zen

Step 1: Check if Zen is Available

Open your Terminal (macOS/Linux) or PowerShell (Windows) and run:

```
opencode models list
```

Look for models labeled with “zen” or “Zen” in the provider column.

Step 2: Connect to Zen

If you see Zen models in the list, you can use them immediately. If prompted to add an API key:

1. Go to <https://opencode.ai/zen> (or visit <https://opencode.ai> and click “Zen”)
2. Sign in with your OpenCode account
3. Navigate to “API Keys” or “Settings”
4. Generate a new API key
5. Copy the key (starts with `zen-` . . . or similar)

Step 3: Configure OpenCode with Zen

In your Terminal/PowerShell, run:

```
opencode configure
```

When prompted: 1. Select “OpenCode Zen” as your provider 2. Paste your Zen API key 3. Choose your preferred model (e.g., `minimax/m2.5` or `moonshotai/kimi-k2.5`) 4. Save the configuration

Step 4: Test Your Connection

Open your Terminal/PowerShell and type:

```
opencode
```

This opens the OpenCode interactive CLI. Once inside, just type:

```
Hello, are you connected to Zen?
```

If you get a response, you’re all set!

Using Zen Models

Once you've configured OpenCode Zen, using the models is simple. Just start OpenCode and begin chatting:

Starting OpenCode

In your Terminal/PowerShell:

```
opencode
```

You'll see the OpenCode prompt appear. Now you can:

1. **Just start typing** - Your default Zen model will respond
2. **Ask anything** - Code questions, debugging help, explanations
3. **Work on files** - OpenCode can read and edit files in your current directory

Example Conversations

Getting help with code:

```
> opencode
```

You: Help me debug this Python error: "IndexError: list index out of range"

OpenCode: Let me help you with that IndexError. This usually happens when...

Working on a project:

```
> opencode
```

You: Read the file main.py and explain what it does

OpenCode: I'll read main.py for you. [Reads file] This file contains...

Switching models (if needed):

You: /model minimax/m2.5

OpenCode: Switched to MinMax M2.5 model.

Free vs Paid Usage

Free Tier (Subject to Change):

- Limited requests per day
- Rate limiting may apply
- Perfect for learning and small projects

Paid Tier:

- Higher rate limits
- Priority access
- Still significantly cheaper than direct API access
- MinMax M2.5: ~\$1/hour of active usage
- Kimi K2.5: Competitive pricing with GPT-4

Tips for Students

1. **Start with free tiers** - Both OpenRouter and Zen offer free access to get you started
2. **Use MinMax M2.5 for coding** - It's specifically optimized for development tasks
3. **Try Kimi K2.5 for multimodal tasks** - Excellent at understanding images and complex reasoning
4. **Monitor usage** - Check your Zen dashboard at <https://opencode.ai/zen> to see usage stats
5. **Combine with local models** - For total privacy, run smaller models locally for free

Managing Your Zen Account

- **Dashboard:** <https://opencode.ai/zen> - View usage, manage keys
- **Add Credits:** Click "Billing" in your Zen dashboard
- **Model Browser:** See available models and their current pricing
- **Community:** Join the OpenCode Discord for Zen-specific tips and updates

Model Provider Options

OpenCode supports multiple model providers:

Provider	Cost	Best For
OpenRouter Free Tier	Free	Getting started, low usage
GitHub Copilot	Subscription	If you already have Copilot
OpenAI (ChatGPT)	Pay-per-use	GPT-4, GPT-3.5
Anthropic (Claude)	Pay-per-use	Claude 3 Opus/Sonnet
Local Models	Hardware-dependent	Complete privacy, no internet

Start with the free tier to explore before committing to paid options.

Understanding Skills

What Are Skills?

Skills are specialized instructions that teach OpenCode domain-specific knowledge. Think of them as custom modules that extend the AI's capabilities for particular tasks, programming languages, or workflows.

Skills are stored as `SKILL.md` files in the `~/.config/opencode/skills/` directory. When you invoke a skill in your prompt, OpenCode loads that skill's instructions to guide its response.

The Anthropic Connection: Why SKILL.md Exists

Anthropic developed the `SKILL.md` format for Claude Code, their AI coding assistant. They recognized that AI agents need structured, reusable instructions to handle complex, domain-specific tasks consistently.

Why Anthropic Created SKILL.md:

1. **Consistency:** Ensures AI agents follow the same patterns across sessions
2. **Reusability:** Define once, use everywhere—across projects and teams
3. **Composability:** Multiple skills can work together on complex tasks
4. **Discoverability:** Agents can see available skills and load them on demand
5. **Safety:** Scoped permissions control which skills agents can access

How OpenCode Uses SKILL.md:

OpenCode adopted `SKILL.md` as a **compatible standard**, meaning:

- Skills written for Claude Code work in OpenCode (and vice versa)
- OpenCode searches multiple locations for maximum compatibility:
 - `.opencode/skills/<name>/SKILL.md` (OpenCode native)
 - `.claude/skills/<name>/SKILL.md` (Claude Code compatible)
 - `.agents/skills/<name>/SKILL.md` (Generic agent compatible)
- The same frontmatter format works across platforms

This compatibility means skills you write today will work with future AI tools that adopt the standard.

How Skills Work

When you write a prompt like:

```
/skill writing-skill Check this paragraph for clarity
```

OpenCode:

1. Discovers available skills from all configured directories
2. Loads the skill file from `~/.config/opencode/skills/writing-skill/SKILL.md`
3. Injects the skill's system instructions into the conversation
4. Processes your request using that specialized knowledge

Skills can include:

- **Frontmatter:** YAML metadata defining name, description, and permissions
- **System instructions:** How the agent should behave when using this skill
- **Code examples:** Reference implementations
- **Best practices:** Domain-specific guidelines
- **Validation rules:** Quality checks to apply

Installing Existing Skills

OpenCode comes with some built-in skills, and you can install community skills from GitHub. Open your Terminal (or PowerShell on Windows) to run these commands:

```
# List available built-in skills
opencode skills list

# Install a skill from a GitHub repository
opencode skills add owner/repo

# Install a local skill
opencode skills add /path/to/skill-directory
```

Example: Installing a Writing Skill

Let's install a writing skill. In your Terminal/PowerShell, run:

```
# Clone a skill repository (hypothetical example)
git clone https://github.com/example/writing-skill.git
↪ ~/.config/opencode/skills/writing-skill
```

After installation, use it within OpenCode (start OpenCode first, then type):

```
/skill writing-skill Review this essay for AI-like phrasing
```

Creating Your First Skill

Skills are simple Markdown files with a specific structure. Let's create a basic skill.

Step 1: Create the Directory

Open your Terminal (or PowerShell on Windows) and run:

```
mkdir -p ~/.config/opencode/skills/my-first-skill
```

Step 2: Write the SKILL.md File

Using your text editor (VS Code, nano, vim, or any text editor), create a file at: `~/.config/opencode/skills/my-first-skill/SKILL.md`

Copy and paste this content (this is the proper YAML frontmatter format):

```
---
name: my-first-skill
description: Help organize project files with structured guidance
license: MIT
compatibility: opencode
metadata:
  category: productivity
  level: beginner
---

## What I do

- Help organize files by type and purpose
- Suggest folder structures for different projects
- Provide clear naming conventions

## When to use me

Use this when you need to organize messy project directories or set up a new
↪ project structure. Ask clarifying questions if the project type or
↪ requirements are unclear.
```

Understanding the Frontmatter Fields

The YAML frontmatter at the top of every SKILL.md file contains metadata that OpenCode uses to discover and manage skills:

Required Fields:

- **name** (required): The skill identifier
 - Must be 1–64 characters
 - Lowercase alphanumeric with single hyphens only
 - Must match the directory name (`my-first-skill` → `my-first-skill/`)
 - Valid: `git-release`, `code-review`, `my-skill`
 - Invalid: `MySkill`, `my--skill`, `-my-skill`, `my_skill`
- **description** (required): A brief explanation of what the skill does
 - Must be 1–1024 characters
 - Should be specific enough for the agent to choose correctly
 - This appears in the skill list when agents query available skills
 - Good: “Review Python code for security vulnerabilities and performance issues”
 - Bad: “A skill for code” (too vague)

Optional Fields:

- **license** (optional): The license under which the skill is distributed
 - Examples: `MIT`, `Apache-2.0`, `GPL-3.0`
 - Helps others understand usage rights
- **compatibility** (optional): Specifies which tools support this skill
 - Examples: `opencode`, `claude`, `agents`
 - Allows cross-tool compatibility
 - Use `opencode` for OpenCode-specific skills
- **metadata** (optional): A string-to-string map for custom key-value pairs
 - Useful for categorization, tagging, or tool-specific features
 - Examples:

```
metadata:
  category: productivity
  level: beginner
  author: your-name
```

- Unknown fields are ignored, so you can add custom tags

Important Rules:

1. **Unknown fields are ignored:** If you add fields not listed above, OpenCode will ignore them without error
2. **YAML syntax matters:** Use proper YAML formatting (2-space indentation, no tabs)
3. **Frontmatter delimiters:** Must start with `---` and end with `---`
4. **Order doesn't matter:** You can list fields in any order within the frontmatter

Step 3: Test Your Skill

Start OpenCode in your Terminal/PowerShell, then type:

```
/skill my-first-skill Help me organize my project files
```

OpenCode will now use your skill's guidelines when processing requests.

Building Effective Skills: Lessons from AI Detection

Creating a good skill requires writing clear, human-like instructions. The Wikipedia article on “Signs of AI Writing” offers valuable lessons for skill authors. Here’s how to apply those insights.

Key Principles from AI Detection Research

The Wikipedia article identifies patterns that make text sound artificial. Skill authors should avoid these:

1. Avoid Vague Abstractions

Problem: AI text often uses phrases like “serves as a testament to” or “underscores the importance of” without specifics.

Solution: Be concrete. Instead of:

```
This skill helps users optimize their code.
```

Write:

```
This skill reduces file processing time by up to 90% through batch  
↔ optimization.
```

2. Skip the “Rule of Three”

Problem: AI loves lists of three: “comprehensive, robust, and scalable.”

Solution: Use the number of items that makes sense. Sometimes two, sometimes five.

3. Don't Overstate Significance

Problem: AI constantly emphasizes importance: “This represents a pivotal moment...”

Solution: State facts plainly. Let the utility speak for itself.

Bad:

```
This skill revolutionizes how you organize files, marking a paradigm shift
↪ in project workflows.
```

Better:

```
This skill helps you organize files faster by using batch operations.
```

4. Use Simple Verbs

Problem: AI avoids simple words like “is” or “has,” using instead “serves as” or “represents.”

Solution: Use direct language.

Bad:

```
This skill serves as a comprehensive guide for file organization best
↪ practices.
```

Better:

```
This skill is a file organization best practices guide.
```

5. Avoid Hedge Phrases

Problem: AI uses vague qualifiers: “It is important to note that...” or “Crucially...”

Solution: Just state the fact.

Structure of an Effective SKILL.md

Based on these principles, here’s a complete template with the required frontmatter:

Required Frontmatter Fields

Each SKILL.md must start with YAML frontmatter. Only these fields are recognized:

- name (required)
- description (required)
- license (optional)
- compatibility (optional)
- metadata (optional, string-to-string map)

Unknown frontmatter fields are ignored.

Complete Template

```
---
name: [skill-name]
description: [Clear explanation of what the skill does]
license: [MIT, Apache-2.0, etc. - optional]
compatibility: [opencode, claude, agents - optional]
metadata:
  [key]: [value]
  [key2]: [value2]
---

## What I do

One clear sentence describing the skill's purpose. No fluff.

## How to use me

Practical instructions. Bullet points if needed (not always 3).

### Core Guidelines

- Be specific about what you check or improve
- Include concrete examples
- Show before/after comparisons where helpful

### Examples

**Input pattern:**
```python
Bad code example here
```

**Output:**

```
Improved code here
```

## What to Avoid

List specific mistakes this skill helps prevent.

### ## Example: A Condensed Writing Skill

Applying these principles, here's a skill for detecting AI writing patterns (condensed from the

```
```markdown
```

```
# Skill: writing-skill
```

What I do

I check writing for AI-style patterns and suggest human alternatives.

How to use me

Give me text to review. I'll flag AI tells and suggest edits.

AI Patterns to Flag

****Vague abstractions:****

- "Serves as a testament to" → "Shows"
- "Underscores the importance of" → "Highlights"
- "Marks a pivotal moment" → "Changed"

****Overused words:****

- Delve, tapestry, landscape, realm, foster, cultivate, leverage
- Intricate, pivotal, vibrant, crucial, seamless, innovative

****Sentence structure tells:****

- "Not only X, but also Y" padding
- Rule of three: "A, B, and C" (always three items)
- False ranges: "From Tokyo to the Alps" when irrelevant

****Formulaic conclusions:****

- "Despite these challenges, [subject] remains..."
- "In summary" at the end of short texts
- "Key takeaways" lists in prose

Human Alternatives

****Instead of:**** "The project serves as a vital tool for developers."

****Use:**** "Developers use this project daily."

****Instead of:**** "This represents a significant shift in the landscape."

****Use:**** "This changed how we work."

****Instead of:**** "Not only is it fast, but also reliable."

****Use:**** "It is fast and reliable."

Specificity Check

I flag vague descriptions and ask for specifics:

- "A variety of options" → How many? Which ones?
- "Many experts agree" → Which experts? How many?
- "Significant impact" → What changed? By how much?

This skill is concise, specific, and gives concrete examples—exactly what makes skills effective.

Best Practices for Skill Development

1. Start Small

Don't try to teach OpenCode everything about a domain. Start with one specific task:

- Good: "Optimize file organization"
- Too broad: "Master project management"

2. Include Concrete Examples

Always show code examples or before/after comparisons. Abstract rules are hard to apply; examples are clear.

3. Focus on What, Not Just How

Explain why a pattern matters:

Bad:

Use vectorized operations.

Better:

Use vectorized operations—they run 100x faster than Python loops because
↳ they execute in C.

4. Version Your Skills

Add a version section to track changes:

```
## Version

- 1.0: Initial release with basic patterns
- 1.1: Added error handling section
```

5. Test Thoroughly

In your Terminal or PowerShell, run your skill through various scenarios:

```
# Test basic functionality
opencode run --skill my-first-skill "Basic task"

# Test edge cases
opencode run --skill my-first-skill "Complex scenario"

# Test error conditions
opencode run --skill my-first-skill "Handle exceptions"
```

6. Iterate Based on Results

Skills improve with use. When OpenCode produces output that misses the mark, update your skill to cover that case.

Advanced Skill Features

Using Multiple Skills

You can combine skills for complex tasks within OpenCode:

```
/skill my-first-skill /skill writing-skill Complete this task and document it clearly
```

(Type this command after starting OpenCode in your Terminal/PowerShell)

Skill Dependencies

Skills can reference other skills. Add to your SKILL.md:

Dependencies

- python-expert (for syntax questions)
- testing-skill (for validation)

Conditional Logic

Advanced skills can include conditional guidance:

When to Apply

```
**If simple task:** Use basic approach  
**If complex task:** Break into subtasks  
**If time-sensitive:** Prioritize speed
```

Troubleshooting

Installation Issues

“Command not found” after installation

```
# Check if binary exists  
ls -la ~/.local/bin/opencode  
  
# Add to PATH manually  
export PATH="$HOME/.local/bin:$PATH"
```

Winget installation fails on Windows

```
# Update WinGet  
winget upgrade --all  
  
# Or use the install script  
curl -fsSL https://opencode.ai/install | bash
```

Skill Loading Issues

“Skill not found” error

```
# Check skill location  
ls ~/.config/opencode/skills/
```

```
# Verify SKILL.md exists
cat ~/.config/opencode/skills/my-skill/SKILL.md
```

Skill doesn't seem to work

- Ensure you're using `/skill skill-name` syntax
- Check that SKILL.md is valid Markdown
- Restart OpenCode to reload skills

Model Connection Issues

“Failed to connect to model”

```
# Test your API key
opencode configure --test

# Check model availability
opencode models list
```

Conclusion

You now have OpenCode installed and understand how to create custom skills. Start with the built-in capabilities, then gradually build your own skills for your specific workflows.

Remember:

- **Skills are teaching tools:** Write clear, specific instructions
- **Avoid AI patterns:** Be concrete, use simple language, skip the fluff
- **Test and iterate:** Skills improve with use
- **Share your skills:** The community benefits from domain-specific expertise

Next Steps

1. Install OpenCode using the method for your OS
2. Configure your model provider
3. Try the built-in skills
4. Create your first SKILL.md using the template in this guide
5. Practice: Convert a repetitive task into a skill

Additional Resources

- **OpenCode Documentation:** <https://opencode.ai/docs>
 - **GitHub Repository:** <https://github.com/anomalyco/opencode>
 - **Discord Community:** <https://opencode.ai/discord>
 - **Model Options:** <https://models.dev>
-

Quick Reference Card

Task	Command
Install (macOS/Linux)	<code>curl -fsSL https://opencode.ai/install \& bash</code>
Install (Windows)	<code>winget install Anomaly.OpenCode</code>
Configure	<code>opencode configure</code>
Use a skill	<code>/skill skill-name your prompt</code>
List skills	<code>opencode skills list</code>
Install skill	<code>opencode skills add owner/repo</code>

Skill Directory: `~/.config/opencode/skills/`